

Department of Computer Science and Engineering, KUET

Subject: Computer Security (CSE 4115)

Implementation of DoS and DDoS Attack Mechanics

A C++ Multi-Threaded Simulation of Denial-of-Service Traffic and Server Resource Exhaustion

---

Submitted by:

Name: Jahid Hasan

Roll: 2107064

Section: B

Year: 4th

Semester: 1st

Submitted to:

Dola Das

Assistant Professor

Department of Computer Science and Engineering

KUET

1 Objectives

For this assignment, I set the following core objectives:

1. **Model Server Resource Exhaustion:** To implement an in-memory simulation of an application server with a bounded FIFO request queue (capacity: 15) and a multi-threaded worker pool (4 concurrent workers with 250 ms processing latency) using C++17 synchronization primitives (`std::mutex`, `std::condition_variable`, `std::atomic`).
2. **Implement a Single-Source DoS Attack:** To simulate an aggressive, high-volume burst flood from a single client origin (`10.0.0.99`) to observe buffer overflow dynamics, queue saturation, and resultant denial-of-service drops for legitimate users.
3. **Implement a Distributed DDoS Botnet Attack:** To model a multi-threaded botnet where distinct concurrent worker threads simulate distributed compromised nodes (across `172.16.x.x` subnets), demonstrating how aggregated concurrent volume overwhelms server capacity and monopolizes worker execution.
4. **Implement Real-Time Telemetry and Health Monitoring:** To construct a decoupled, non-blocking monitoring engine running on an independent thread (`serverMonitor`) that periodically aggregates windowed arrival and drop metrics, computes real-time drop rates, inspects worker busy/idle states, and tracks legitimate service availability percentages.
5. **Maintain a Pure, Safe In-Memory Architecture:** To formulate the complete simulation in clean, standard C++ without requiring external network socket frameworks, raw packet craft libraries, or targeting physical network interfaces, ensuring safe execution on local machines.

2 Introduction

A Denial of Service (DoS) attack aims to disrupt the availability of a service by exhausting critical server resources such as CPU time, worker threads, or incoming connection buffers. In a basic DoS scenario, all malicious traffic originates from a single client machine.

A Distributed Denial of Service (DDoS) attack accomplishes the same goal but coordinates traffic across multiple compromised hosts (a botnet). Because individual bots can send requests at modest, seemingly normal rates, basic per-IP rate-limiting techniques struggle to identify them, even though their collective traffic saturates the target server.

To explore these dynamics safely, I designed an in-memory concurrent simulation in C++ using a synchronized bounded FIFO queue (`std::queue`), worker threads (`std::thread`), mutexes (`std::mutex`), and condition variables (`std::condition_variable`). This allowed me to accurately model queue backlogs, thread pool starvation, arrival rate spikes, and request drop behavior entirely within software memory.

3 Code Implementation & Explanation

I developed the entire simulation within a single file named `dos_ddos_simulator.cpp`. Below, I explain the architecture, data structures, and algorithms I used.

3.1 Data Modeling: The Request Origin and Request Structs

To represent incoming traffic and worker activity, I created structured data types. The `RequestOrigin` enum categorizes the source, while `Request` stores a unique identifier, origin tag, client IP string, and high-resolution arrival timestamp. The `WorkerStatus` struct tracks active worker allocations:

```
enum class RequestOrigin { LEGITIMATE, DOS_BURST, DDOS_BOTNET };
struct Request {
    uint64_t id; RequestOrigin origin; string clientIp;
    chrono::steady_clock::time_point arrivalTime;
};
struct WorkerStatus {
    bool isBusy{false}; uint64_t currentRequestId{0};
    RequestOrigin currentOrigin{RequestOrigin::LEGITIMATE}; string currentClientIp{""};
};
```

3.2 Server Architecture: The ApplicationServer Class

I encapsulated the server logic inside the `ApplicationServer` class. It maintains four essential components:

- A bounded FIFO buffer (`requestQueue`) with a fixed maximum capacity of 15.
- A worker thread pool (4 concurrent workers) that simulates real-world processing latency (`processingTime = 250ms`).
- Thread-safe synchronization primitives: `queueMutex`, `statusMutex`, and condition variable `cvQueue`.
- Atomic telemetry counters (`std::atomic<uint64_t>`) for thread-safe traffic and drop accounting.

```
class ApplicationServer {
private:
    const size_t numWorkers; const size_t maxQueueCapacity;
    const chrono::milliseconds processingTime;
    vector<thread> workers; vector<WorkerStatus> workerStatuses;
    queue<Request> requestQueue;
    mutex queueMutex, statusMutex; condition_variable cvQueue;
    atomic<bool> isRunning{true};
    atomic<uint64_t> totalLegitReceived{0}, totalLegitProcessed{0}, totalLegitDropped{0};
    atomic<uint64_t> totalAttackReceived{0}, totalAttackProcessed{0}, totalAttackDropped{0};
public:
    ApplicationServer(size_t workersCount, size_t queueCapacity, int serviceTimeMs)
        : numWorkers(workersCount), maxQueueCapacity(queueCapacity),
          processingTime(serviceTimeMs), workerStatuses(workersCount) {}
    void start(); void stop(); bool submitRequest(const Request &req);
};
```

3.3 Consumer Engine (Multi-Threaded Worker Pool Loop)

To model a server executing requests (such as database queries or dynamic rendering), I implemented a background worker loop inside `workerLoop()`. Each worker waits efficiently on `cvQueue`, pops an incoming request, pauses for `processingTime` (250 ms) to simulate execution, and updates worker telemetry:

```
void workerLoop(size_t workerId) {
    while (isRunning) {
        Request currentReq;
        {
            unique_lock<mutex> lock(queueMutex);
            cvQueue.wait(lock, [this]() { return !requestQueue.empty() || !isRunning; });
            if (!isRunning && requestQueue.empty()) break;
            if (requestQueue.empty()) continue;
            currentReq = requestQueue.front(); requestQueue.pop();
        }
        {
            lock_guard<mutex> lock(statusMutex);
            workerStatuses[workerId] = {true, currentReq.id, currentReq.origin, currentReq.clier
        }
        this_thread::sleep_for(processingTime); // Simulates 250ms service workload
        if (currentReq.origin == RequestOrigin::LEGITIMATE) totalLegitProcessed++;
        else totalAttackProcessed++;
        {
            lock_guard<mutex> lock(statusMutex);
            workerStatuses[workerId].isBusy = false;
        }
    }
}
```

3.4 Request Ingestion and Drop Logic

I implemented the entry point for all incoming traffic in `submitRequest()`. This function handles thread-safe admission control and bounded queue capacity checks:

```
bool submitRequest(const Request &req) {
    if (req.origin == RequestOrigin::LEGITIMATE) totalLegitReceived++;
    else totalAttackReceived++;
    {
        unique_lock<mutex> lock(queueMutex);
        if (requestQueue.size() >= maxQueueCapacity) {
            // Buffer capacity exceeded (15 slots): drop request
            if (req.origin == RequestOrigin::LEGITIMATE) totalLegitDropped++;
            else totalAttackDropped++;
            return false;
        }
        requestQueue.push(req);
    }
    cvQueue.notify_one();
    return true;
}
```

Design Rationale:

- I enforced a non-blocking rejection mechanism when the queue reaches 15 items. If full, the incoming request is immediately dropped and accounted for, without blocking the sender. This reproduces how real operating systems drop incoming packets when the TCP socket listen backlog is saturated.
- I used `cvQueue.notify_one()` so that only a single waiting worker is woken per accepted request, avoiding CPU cache thrashing and context-switch overhead ("thundering herd" prevention).

3.5 Real-Time Telemetry and Monitoring Engine

To monitor system health under load without introducing lock contention, I constructed a decoupled telemetry dashboard inside `printDashboard()`. Executed periodically (every 2.0 seconds) by an independent `serverMonitor` thread, it samples metrics and computes windowed drop rates:

```
void printDashboard(double intervalSeconds) {
    uint64_t curLegitDrop = totalLegitDropped.load();
    uint64_t deltaLegitDrop = curLegitDrop - lastLegitDropped; lastLegitDropped = curLegitDrop;
    double legitDropRate = static_cast<double>(deltaLegitDrop) / intervalSeconds;
    double legitSuccessPct = (curLegitRecv > 0) ?
        (100.0 * (curLegitRecv - curLegitDrop) / curLegitRecv) : 100.0;
    // Displays worker states, ASCII queue capacity meter [####....], and health stats
}
```

3.6 Implementation of the DoS Attack Scenario

To simulate a single-source DoS attack, I wrote `launchDosBurst()`. In this function, an attacker host at IP `10.0.0.99` floods a rapid burst of requests (e.g., 40 requests) in a tight loop into the server:

```
void launchDosBurst(ApplicationServer &server, int count) {
    string attackerIp = "10.0.0.99 (Attacker-Host, simulated)";
    for (int i = 0; i < count; ++i) {
        Request req{gRequestIdCounter++, RequestOrigin::DOS_BURST,
                    attackerIp, steady_clock::now()};
        server.submitRequest(req);
    }
}
```

Observation: Because the arrival rate of the burst is instantaneous while the 4 worker threads drain requests at a combined maximum rate of approximately 16 requests/second (4 workers / 0.25s), the bounded 15-slot queue fills to capacity almost immediately. In a burst of 40 requests, 25 requests were dropped due to buffer overflow. Any legitimate requests arriving during this saturated window are denied service.

3.7 Implementation of the DDoS Attack Scenario

To model a Distributed Denial of Service (DDoS) attack, I created `launchDdosFlood()`. Here, I simulated a multi-source botnet of 10 concurrent bot threads. Each bot generates traffic from a distinct IP address across the `172.16.x.x` subnet and transmits 5 requests:

```
void launchDdosFlood(ApplicationServer &server, int botCount, int reqsPerBot) {
    vector<thread> botThreads; botThreads.reserve(botCount);
    for (int b = 0; b < botCount; ++b) {
        botThreads.emplace_back([&server, b, reqsPerBot]() {
            string botIp = "bot-" + to_string(b + 1) + ".botnet.example (" +
                to_string(172) + "." + to_string(16 + (b % 16)) + "." +
                to_string(b / 254) + "." + to_string(b % 254 + 1) + ")";
            for (int r = 0; r < reqsPerBot; ++r) {
                Request req{gRequestIdCounter++, RequestOrigin::DDOS_BOTNET,
                            botIp, steady_clock::now()};
                server.submitRequest(req);
                this_thread::sleep_for(chrono::microseconds(500));
            }
        });
    }
    for (auto &th : botThreads) if (th.joinable()) th.join();
}
```

Observation: In the DDoS scenario, requests arrive concurrently from multiple threads. All 4 workers become saturated simultaneously processing attack traffic, and the shared queue rapidly reaches full capacity (15/15), leading to heavy packet drops. Because traffic originates from numerous distinct IP addresses, simple single-IP rate-limiting or blacklisting fails to mitigate the attack.

3.8 Background Normal Traffic Generation

To establish a realistic baseline, I implemented `legitimateTrafficGenerator()` on a dedicated thread. It generates legitimate requests with randomized client IPs (`192.168.1.10` to `192.168.1.99`) spaced every 280 ms (~3.57 req/sec):

```
void legitimateTrafficGenerator(ApplicationServer &server) {
    mt19937 rng(1337); uniform_int_distribution<int> ipSuffix(10, 99);
    while (gSimRunning) {
        this_thread::sleep_for(chrono::milliseconds(280));
        if (!gSimRunning) break;
        Request req{gRequestIdCounter++, RequestOrigin::LEGITIMATE,
                    "192.168.1." + to_string(ipSuffix(rng)), steady_clock::now()};
        server.submitRequest(req);
    }
}
```

3.9 Experimental Results and Verification

When I executed the simulation binary (`dos_sim.exe`), I observed the following telemetry outputs across the three testing phases:

```
--- [PHASE 1: NORMAL BASELINE WORKLOAD] ---
WORKER POOL STATUS: Worker #0: [IDLE] | Worker #1: [BUSY] Req #1006 (LEGIT) from 192.168.1.51
REQUEST QUEUE CAPACITY: [0 / 15] [.....................]
Normal User Requests : Received: 7 | Dropped: 0 | DROP RATE: 0.00 req/sec
Normal Service Health: 100.0% Availability

--- [PHASE 2: SINGLE-SOURCE DoS ATTACK (40 Requests)] ---
[!] SIMULATING DoS ATTACK: single-source burst of 40 requests...
REQUEST QUEUE CAPACITY: [15 / 15] [####################] (Buffer Saturated)
Attack Requests: Received: 40 | Dropped: 25 | Total Attack Requests Dropped: 25

--- [PHASE 3: DISTRIBUTED DDoS BOTNET FLOOD (10 Bots x 5 Reqs = 50 Reqs)] ---
[!] SIMULATING DDoS ATTACK: 10 simulated botnet nodes sending 5 requests each...
WORKER POOL: All 4 Workers [BUSY] processing bot traffic (bot-1, bot-2, bot-3, bot-4)
Cumulative Server Health: Total Attack Requests: 90 (Processed: 29, Dropped: 57)
```

Scenario	Traffic Injected	Worker State	Max Queue	Dropped Reqs	Service Impact
Normal Workload	~3.5 req/sec	1 Busy, 3 Idle	0 / 15	0 dropped	100.0% Availability
Single-Source DoS	40 burst (1 IP)	4 Busy	15 / 15 (Full)	25 dropped	Buffer overflow; drop spikes
Distributed DDoS	50 concurrent (10 IPs)	4 Busy (100%)	15 / 15 (Full)	32 dropped	Resource exhaustion; thread lock

4 Mitigation Strategies (Discussion)

- **Per-IP Rate Limiting (Token Bucket):** Caps maximum submission frequency per source IP. Highly effective for mitigating single-source DoS bursts, but ineffective against distributed botnets where each bot remains below the per-IP threshold.
- **SYN Cookies & Connection Backlogs:** Prevents embryonic connection table starvation by deferring state allocation until full three-way TCP handshake completion.
- **Priority Queuing & Client Authentication:** Segregates known authenticated user requests into dedicated high-priority queues, preventing anonymous floods from starving legitimate users.
- **Edge Scrubbing & Anycast CDNs:** Utilizing upstream distributed scrubbing centers (e.g., Cloudflare, AWS Shield) to absorb volumetric floods before they reach origin application servers.

5 Conclusion

This lab demonstrated the fundamental vulnerability of shared server resources to volumetric and connection-queue exhaustion attacks. Through this multi-threaded C++ simulation, I showed how buffer overflow causes service denial, how worker thread starvation degrades throughput, and why distributed attacks pose significantly greater mitigation challenges than single-source floods.